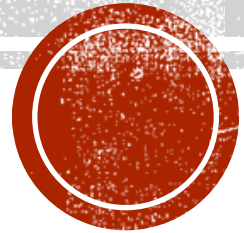
COS 202- COMPUTER PROGRAMMING II

MISS OYEWOLE

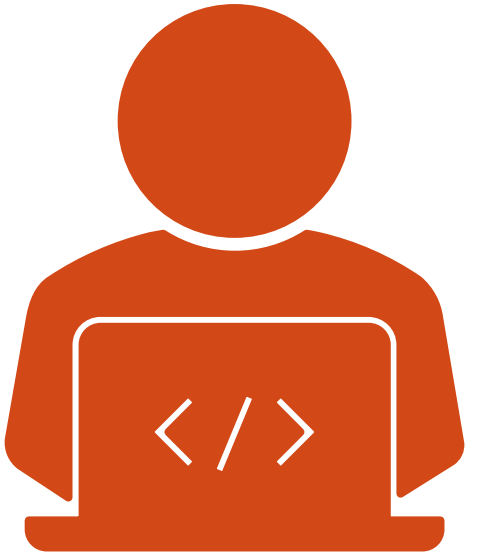


JAVA OBJECT ORIENTED PROGRAMMING CONCEPTS

Java is an Object-Oriented Programming language which has a similar syntax to C/C++ programming languages but without its complexities. Java is widely used to develop mobile applications, desktop applications, web applications, web servers, games, and enterprise-level systems. Originally invented by James Gosling, Java is now owned by Oracle, with JDK 23 being its latest version. The language features a syntax similar to that of C and C++, making it familiar to many developers. Renowned platforms such as LinkedIn, Amazon, and Netflix utilize Java for their back-end systems, highlighting its reliability, scalability, and adaptability across various environments.

OBJECT-ORIENTED PROGRAMMING CONCEPTS

Object-oriented programming (OOP) involves programming using objects. **Object-Oriented Programming** is a methodology to design a program using classes and objects. **An object** represents an entity in the real world that can be distinctly identified. **Object** means a real-world entity such as a pen, chair, table, computer, watch, etc. An object has a unique identity which include; state and behaviour.



- The state of an object (also known as its properties or attributes) is represented by data fields with their current values. A circle object, for example, has a data field radius, which is the property that characterizes a circle.
- The behaviour of an object (also known as its actions) is defined by methods. To invoke a method on an object is to ask the object to perform an action. For example, you may define a method named getArea() for circle objects. A circle object may invoke getArea() to return its area.

BASIC PRINCIPLES OF OOP (OBJECT-ORIENTED PROGRAMMING)

- **Class** - A class is a template or blueprint from which objects are created. It describes a set of objects with the same behaviour. A class can contain Fields, methods, constructors, blocks, nested class and interface. To create a class, you use the keyword *class.*

```
public class Major {

Int x = 10;

}
```

- **Object** - An object is an instance of a class. It is an entity that has state and behaviour. It can be physical or logical (tangible and intangible).

```
public class Main {

int x = 5;


public static void main(String[] args) {

Main myObj = new Main();

System.out.println(myObj.x);

}

}
```

- **Abstraction** - The process of hiding the details and exposing only the essential features of a particular concept or object without including the background details or explanation.



BASIC PRINCIPLES OF OOP (OBJECT-ORIENTED PROGRAMMING)

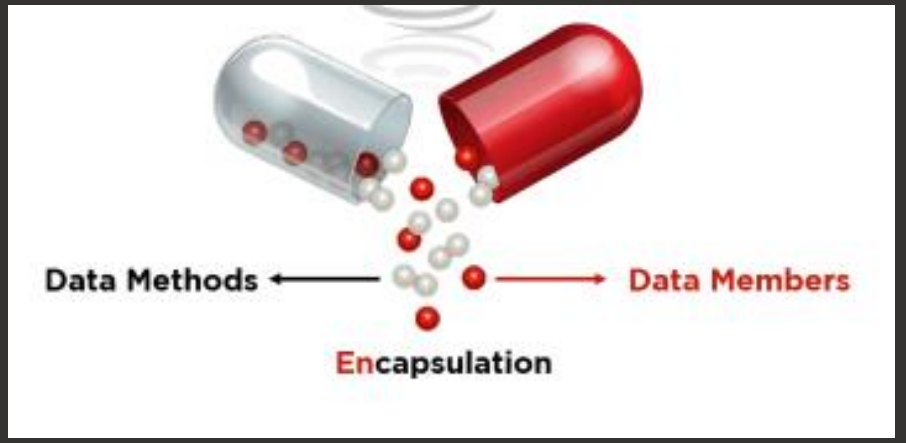
- **Inheritance** - Inherit members from parent class. Inheritance is the process of building new classes based on parent classes. The new class inherits the properties and attributes of the parent class
- **Polymorphism**- means the ability to take more than one form. An operation may exhibit different behaviour in different instances. The behaviour depends upon the types of data used in the operation. In java, we use **method overloading** and **method overriding** to achieve polymorphism.
- **Encapsulation** - Hide the internals of a class. This is wrapping up of data and functions into a single unit (called class).



JAVA ENCAPSULATION

Encapsulation in Java refers to integrating data (variables) and code (methods) into a single unit. In encapsulation, a class's variables are hidden from other classes and can only be accessed by the methods of the class in which they are found.

Difference between **encapsulation** and **data hiding** is that Encapsulation is a way of bundling data whereas Data Hiding prevents external unauthorized access to the sensitive data. Access modifiers are used to achieve data hiding. Access modifiers are the keywords that specify the accessibility or the scope of methods, classes, fields, and other members



TYPES OF ACCESS MODIFIERS IN JAVA

- **Public:** A public modifier is accessible to everyone. It can be accessed from within the class, outside the class, within the package, and outside the package.
- **Private:** A private modifier can not be accessed outside the class. It provides restricted access.
- **A protected modifier:** can be accessed from the classes of the same package and outside the package only through inheritance.
- **A default modifier** is accessible only within the package. If no modifier is used, it is treated as default.



HOW TO IMPLEMENT ENCAPSULATION IN JAVA

There are two steps to achieve the purpose of Encapsulation in Java which are;

- Use the private access modifier to declare all variables/fields of class as private
- Define public getter and setter methods to read and modify/set the values of the above said fields

```
public class Person {
private String name; // private = restricted access
// Getter
public String getName() {
return name;
}
// Setter
public void setName(String newName) {
this.name = newName;
}
}
```



BENEFITS OF JAVA ENCAPSULATION

- Cleaner, more organized and less complex code.
- Makes the code more secure.
- The code can be maintained at any point without breaking the classes that use the code.
- Better control of class attributes and methods
- Flexible: the programmer can change one part of the code without affecting other parts
- Increased security of data





Package in Java is a mechanism to encapsulate a group of classes, sub packages and interfaces. Packages are used for:

- Preventing naming conflicts. For example, there can be two classes with name Employee in two packages, college.staff.cse.Employee and college.staff.ee.Employee
- Also define class members that are only exposed to other members of the same package
- Making searching/locating and usage of classes, interfaces, enumerations and annotations easier
- Providing controlled access: protected and default have package level access control. A protected member is accessible by classes in the same package and its subclasses. A default member (without any access specifier) is accessible by classes in the same package only.
- Packages can be considered as data encapsulation

WHY JAVA?

- **Portability:** Java is a highly portable programming language because it is not designed for any specific hardware or software platform
- **Secure:** Java is a very secure programming language. Java codes (applets) may not access the memory on the local computer that they are downloaded upon. Thus, it provides a secure means of developing internet applications.
- **Simple:** Java's feature makes it a concise programming language that is easy to learn and understand.
- **Robustness:** Java is a strongly typed programming language and encourages the development of error free applications.



TYPES OF JAVA PROGRAMS

Java programs can be developed in three different ways, which are briefly outlined here;

- **Java Applications:** These are stand-alone applications such word processors, inventory control systems etc.
- **Java Applets:** is a Java program that can be embedded into a web page. It runs inside the web browser and works at client side. These programs that are executed within a browser. They are executed on the client computer.
- **Java Serverlets:** the Java programs that run on the Java-enabled web server or application server. They are used to handle the request obtained from the web server, process the request, produce the response, and then send a response back to the web server. These are server-side programs that are executed within a browser.

